

REDr

SaaS de Gestion de Location Automobile

MINI-CAHIER DES CHARGES

Livrable S2 — Module Méthodologies de Développement Logiciel

Projet	REDr — SaaS de Gestion de Location Automobile
Établissement	Faculté des Sciences — Université Mohammed V, Rabat
Encadrante	Pr. Sanae EL MIMOUNI
Année universitaire	2025 — 2026
Version	v1.0 — Mars 2026
Équipe	Ali Elkhayatte (Chef de projet) · Achraf Bejjih (Product Owner) · Mohamed Ettalbi (Développeur / Testeur)

Stack : Spring Boot + React · PostgreSQL · DigitalOcean · JWT
Méthodologie : Scrum / MoSCoW · Jalons S1–S12

1. Contexte et Problématique

1.1 Situation actuelle

La location automobile au Maroc est un secteur en forte croissance avec plus de 50 000 véhicules, mais la majorité des agences locales utilise encore des méthodes manuelles ou semi-manuelles comme Excel non synchronisé et des contrats manuscrits. Les réservations sont souvent notées dans un cahier ou un fichier partagé, ce qui cause plusieurs fois des doubles réservations. La disponibilité des véhicules dépend de la mémoire du gérant et des employés. Les contrats manuscrits prennent 15 à 20 minutes, ce qui fait perdre du temps. Il n'y a pas toujours de photos pour l'état des lieux, ce qui crée des conflits difficiles à prouver. De plus, les clients ne peuvent pas réserver en dehors des heures d'ouverture, ce qui entraîne la perte de 3 à 5 clients par semaine et environ 300 heures par an.

1.2 Problème résolu

REDr résout le problème en permettant aux agences indépendantes de gérer efficacement leur flotte, leurs réservations et leurs contrats, tout en offrant aux clients une expérience de réservation en ligne moderne, disponible 24h/24, sans investissement technique ni formation longue.

1.3 Valeur ajoutée chiffrée

Problème identifié	Impact actuel	Gain REDr
Appels de disponibilité	~300 heures / an	Éliminé (portail 24h/24)
Clients perdus hors heures	3 à 5 / semaine	Réservation automatique
Litige sans état des lieux	~2 800 DH / incident	Photos horodatées probantes
Traitement d'un contrat	15–20 minutes	< 3 minutes (PDF auto)

2. Parties Prenantes

Le tableau ci-dessous identifie les cinq parties prenantes du projet REDr, leurs besoins spécifiques et leur niveau d'implication.

Rôle	Besoins principaux	Implication	Priorité
Gérant d'agence (B2B)	Gérer flotte, réservations, contrats en temps réel depuis mobile	Utilisateur principal	Critique
Client final / MRE (B2C)	Réserver un véhicule en ligne 24h/24 sans appel téléphonique, recevoir confirmation	Utilisateur secondaire	Critique
Super Administrateur	Valider les comptes agences, surveiller la plateforme, accéder aux statistiques globales	Équipe REDr	Élevée
Employé d'agence	Consulter la disponibilité, créer des réservations sans dépendre du gérant	Utilisateur opérationnel	Moyenne

3. Périmètre — Méthode MoSCoW

Les fonctionnalités sont classées selon la méthode MoSCoW. Les 3 Must Have constituent le cœur du MVP livrable en S12.

Priorité	Réf.	Fonctionnalité	Justification
Must Have	F1	Authentification multi-rôles sécurisée (Client / Agence / Admin) avec JWT et validation de compte	Prérequis technique de toutes les autres fonctionnalités
Must Have	F2	Gestion de flotte en temps réel (CRUD véhicules, tableau de bord code couleur, statuts dynamiques)	Cœur du produit — sans flotte, rien ne fonctionne
Must Have	F3	Système de réservation — portail client 24h/24, calendrier, génération contrat PDF, état des lieux numérique	Promesse principale — remplace Excel et appels
Should Have	F4	Espace client — historique réservations, téléchargement contrats PDF, annulation avec politique affichée	Améliore l'expérience client et réduit les appels
Could Have	F5	Paiement en ligne (CMI) rappels de retour de véhicule, alertes maintenance programmée	Intégré si Must/Should livrés avant S11
Could Have	F6	Tableau de bord statistiques agence — taux d'occupation, revenus, graphiques Recharts, export CSV	Valeur analytique — non bloquant pour S12
Won't Have v1	—	Rappels de retour de véhicule, alertes maintenance programmée, application mobile native iOS/Android, messagerie temps réel, comparateur multi-agences, API publique	Hors du réglementaire du MVP

4. Contraintes

4.1 Contraintes techniques — Stack imposée

- ▶ Chaque requête Spring Boot vérifie l'identité de l'agence (claim JWT) avant tout accès PostgreSQL — aucune agence ne peut accéder aux données d'une autre.
- ▶ Garantie ACID — deux réservations simultanées sur le même véhicule entraîne le rejet automatique de l'une des deux.
- ▶ Interface React mobile-first — toutes les fonctionnalités critiques accessibles sur écran 375px minimum.
- ▶ Pipeline CI/CD opérationnel dès S10 — Docker + GitHub Actions : builds automatiques, tests d'intégration avant tout déploiement sur DigitalOcean.
- ▶ Disponibilité de 99,5 % dans les périodes de haute saison juillet-août, avec monitoring DigitalOcean et alertes si le CPU > 80 % et aussi informe les agences clairement à l'avance des maintenances prévues.

4.2 Contraintes de délai

Le MVP du projet REDr est plus grand que un projet qui peut durer près de 12 semaines. Donc nous décidons de faire un paiement fictif F5 pour dérouler l'application web et aussi on va annuler F6 qui est le tableau de bord statistiques agence.

4.3 Contraintes de sécurité et légales

- ▶ HTTPS obligatoire sur tous les endpoints Spring Boot en production
- ▶ Conformité Loi 09-08 (protection des données personnelles au Maroc) : consentement explicite, droit d'accès, droit de suppression, conservation des contrats 5 ans minimum
- ▶ Isolation multi-tenant stricte : chaque agence ne peut accéder qu'à ses propres données (filtre agence_id sur chaque requête)
- ▶ Blocage du compte après 5 tentatives de connexion échouées + email d'alerte

5. Dépendances Externes

Service / API	Usage dans REDr	Risque si indisponible
SendGrid (email)	Envoi confirmations réservations, relances créances, activation de compte	Notifications bloquées — réservations créées sans email de confirmation. Fallback : log en base et retry scheduler
Cloudinary	Stockage photos véhicules et photos état des lieux horodatées	Upload photos impossible — fonctionnalité état des lieux désactivée. Fallback : stockage local temporaire
GitHub Actions (CI/CD)	Pipeline build + tests + déploiement automatique sur DigitalOcean	Déploiement manuel requis — livraison S10 retardée, risque qualité
PostgreSQL (BDD)	Transactions ACID, verrouillage SELECT FOR UPDATE, stockage de toutes les données	Application hors service — risque critique, nécessite monitoring DigitalOcean avec alertes automatiques
Spring Security / JWT	Authentification stateless multi-rôles, invalidation tokens à la suspension	Aucun accès à la plateforme — bibliothèque interne, pas de dépendance externe directe

6. Cas d'Utilisation

UC-01 : Inscription et connexion

Élément	Description
Acteur	Gérant d'agence (ROLE_AGENCE), Client final / MRE (ROLE_CLIENT)
Précondition	L'utilisateur n'est pas encore inscrit sur REDr. L'utilisateur accède au portail React REDr depuis un navigateur, dispose d'une adresse email valide, et n'est pas encore inscrit.
Déroulement	1. L'utilisateur remplit le formulaire React (email, mot de passe ≥8 car., autre infos, CGU + Loi 09-08). 2. Spring Boot vérifie l'unicité de l'email. 3. Compte créé avec statut EN_ATTENTE_VALIDATION. 4. Email de confirmation envoyé via SendGrid. En cas Gérant d'agence : 5. Super Admin valide le compte depuis l'interface admin React. 6. Email d'activation envoyé au gérant. En cas Client final / MRE :

	5. Le client clique le lien et passe le compte à ACTIF. 6. Affichage l'espace client.
Résultat attendu	L'utilisateur peut se connecter et accéder au son interface avec son role (ROLE_AGENCE/ ROLE_CLIENT).
Cas d'erreur	Email déjà utilisé → message d'erreur React. Mot de passe trop faible → validation côté React avant envoi. En cas Gérant d'agence : Super Admin refuse → email de refus avec motif obligatoire (min 20 caractères).

UC-02 : Valider et gérer les comptes agences

Élément	Description
Acteur	Super Administrateur (ROLE_ADMIN)
Précondition	Le Super Admin est authentifié avec ROLE_ADMIN, JWT valide.
Déroulement	1. Le Super Admin accède l'interface d'administration qui affiche la liste de toutes les agences avec statuts depuis PostgreSQL avec filtres. 2. Le Super Admin clique sur le compte EN_ATTENTE, examine les informations, clique 'Valider' ou 'Refuser' avec saisit le motif. 3. Pour suspendre un compte ACTIF : Le Super Admin saisit motif et durée de suspension.
Résultat attendu	Le Super Admin consulte les demandes d'inscription agences, valide ou refuse les nouveaux comptes, et suspend ou supprime des comptes existants depuis l'interface d'administration React.
Cas d'erreur	Suspension d'une agence avec réservations CONFIRMEES actives → le système indique le nombre de réservations actives et affiche une alerte, puis l'admin peut confirmer en connaissance de cause. Suppression définitive → le système vérifie s'il n'y a plus de réservations actives ni de contrats < 5 ans ; sinon, la suppression est refusée et seules les données personnelles sont anonymisées.

UC-03 : Réservation en ligne par un client (24h/24)

Élément	Description
Acteur	Client final / MRE (ROLE_CLIENT)
Précondition	Le client est inscrit et connecté. Au moins un véhicule de l'agence est au statut DISPONIBLE pour les dates souhaitées.
Déroulement	1. Client saisit dates, ville, catégorie → API Spring Boot calcule disponibilités en PostgreSQL. 2. Client sélectionne un véhicule, consulte la fiche (photos, prix, conditions). 3. Client confirme la réservation avec Paiment → Spring Boot exécute SELECT FOR UPDATE pour verrouiller le véhicule. 4. Génération contrat PDF automatique avec statut EN_ATTENTE. 5. Email de validation avec contrat envoyé < 5 secondes via SendGrid.
Résultat attendu	Réservation créée, confirmation email reçue, véhicule bloqué pour les dates sélectionnées. Aucune double réservation possible par design (ACID).
Cas d'erreur	Conflit de dates (double réservation simultanée) → Spring Boot rejette la seconde avec HTTP 409 et message explicite. Véhicule en maintenance → réservation impossible, message d'erreur React.

UC-04 : Créer les réservations (vue agence)

Élément	Description
Acteur	Gérant d'agence (ROLE_AGENCE)
Précondition	Le gérant est authentifié avec ROLE_AGENCE, JWT valide. Au moins un véhicule au statut DISPONIBLE existe.
Déroulement	1. Le gérant accède à 'Réservations'. 2. clique <<+ Nouvelle réservation>>, React affiche le formulaire(saisit les informations du client si il est nouveau; sinon recherche et sélectionne le client existant, choisie les dates de départ et de retour, sélectionne la vehicule). 3. Gérant confirme la réservation → Spring Boot exécute SELECT FOR UPDATE pour verrouiller le véhicule. 4. Génération contrat PDF automatique avec statut EN_ATTENTE. 5. Email de validation avec contrat envoyé < 5 secondes via SendGrid au client.
Résultat attendu	Le gérant consulte, crée, ou annule les réservations de son agence. Il crée des réservations directement pour les clients se présentant en agence ou appelant par téléphone.
Cas d'erreur	Conflit de dates (double réservation simultanée) → Spring Boot rejette la seconde avec HTTP 409 et message explicite. Véhicule en maintenance → réservation impossible, message d'erreur React.

7. Architecture Technique

REDr repose sur une architecture 3 couches classique, intégralement justifiée par les contraintes terrain et académiques.

Couche	Technologie	Justification
Frontend	React + Vite + React Router + Axios	Interfaces dynamiques (hooks, état), mobile-first 375px, composants réutilisables. Gérants et clients accèdent depuis smartphone en mobilité.
Backend	Spring Boot (Java) + Spring Security + Spring Data JPA	API REST robuste, annotations @PreAuthorize par rôle, transactions ACID, gestion multi-tenant par filtre agence_id, SELECT FOR UPDATE anti-doublon.
Base de données	PostgreSQL + index sur vehicle_id / dates	SGBD relationnel ACID pour les contraintes de réservation. Index optimisés sur les plages de dates — requête disponibilité < 100ms.
Hébergement	DigitalOcean App platform	Déploiement continu via la plateforme, supervision intégrée des performances (CPU, mémoire) et gestion simplifiée de l'infrastructure, avec un coût maîtrisé d'environ 6 \$/mois adapté à un MVP.
Services tiers	SendGrid (email) + Cloudinary (stockage photos)	Notifications transactionnelles gratuites (100/j), stockage photos état des lieux avec URL signées 15 min pour valeur probante.

Flux de données : Client/Gérant → React (HTTPS) → API Spring Boot (JWT vérifié) → PostgreSQL (requête filtrée par `agence_id`) → Réponse JSON → React (mise à jour état).

8. Analyse des Risques

ID	Risque	Probabilité	Impact	Mitigation
RT-01	Double réservation d'un même véhicule (conflit dates)	Élevée	Critique	SELECT FOR UPDATE PostgreSQL — ACID garanti
RT-02	Difficulté d'implémentation génération PDF contrat	Moyenne	Élevé	POC obligatoire avant S6 — solution de repli OpenPDF
RT-03	Complexité architecture multi-tenant Spring Boot + PostgreSQL	Moyenne	Élevé	Architecture définie Sprint 0 — tests isolation HTTP 403
RO-01	Code mal structuré dès le départ (dette technique)	Moyenne	Élevé	Revue de code croisée obligatoire avant chaque merge
RO-02	Sous-estimation complexité — dépassement délai académique S12	Élevée	Élevé	MoSCoW strict, vitesse Jira surveillée chaque séance

9. Glossaire

Terme	Définition dans le contexte REDr
Multi-tenant	Architecture permettant à plusieurs agences d'utiliser la même instance REDr tout en maintenant une isolation stricte de leurs données via le filtre <code>agence_id</code> .
SELECT FOR UPDATE	Instruction PostgreSQL qui verrouille une ligne pendant une transaction pour éviter les conflits de réservation simultanée — mécanisme central de REDr contre les doublons.
JWT	JSON Web Token — jeton d'authentification stateless généré par Spring Boot à la connexion, contenant le rôle de l'utilisateur et l'identifiant de son agence.
ROLE_AGENCE	Rôle Spring Security attribué au gérant et employés d'agence — donne accès au tableau de bord, à la gestion de flotte, des réservations et des créances.
MRE	Marocain Résidant à l'Étranger — profil client clé qui réserve depuis l'Europe, hors horaires d'ouverture. Motivé le choix de la disponibilité 24h/24.
MVP	Minimum Viable Product — version de REDr livrable en S12 incluant F1, F2, F3 (Must Have) et si possible F4/F4+ (Should Have), déployée sur DigitalOcean.

Loi 09-08	Loi marocaine sur la protection des données personnelles. REDr l'implémente via consentement explicite à l'inscription, droit d'accès/suppression et conservation des contrats 5 ans.
------------------	---

10. Hypothèses et Exclusions

10.1 Hypothèses — Ce qu'on suppose vrai

- Les gérants cibles disposent d'un smartphone récent avec connexion internet (4G minimum) — condition validée lors des entretiens terrain.
- Les agences partenaires acceptent de payer un abonnement fixe de 400–600 DH/mois sans commission sur les réservations — modèle validé en entretien.
- Les clients finaux (notamment MRE) disposent d'une adresse email valide et savent effectuer une réservation en ligne — public cible technophile.
- La plateforme DigitalOcean reste opérationnelle avec une disponibilité de 99,9 % — garantie SLA DigitalOcean.
- L'équipe dispose des licences étudiantes gratuites pour IntelliJ, GitHub et les outils nécessaires au développement.

10.2 Exclusions — Ce que REDr v1 ne fait PAS

- Paiement en ligne intégré (agrément Bank Al-Maghrib requis pour le MVP) — fake paiement pour dérouler l'application web.
- Application mobile native iOS/Android — l'interface React mobile-first 375px couvre les besoins terrain sans les coûts de développement natif.
- Messagerie temps réel agence-client — les emails transactionnels via SendGrid couvrent les besoins de communication identifiés.
- Comparateur multi-agences / portail OTA — REDr est un SaaS de gestion B2B, pas un comparateur public.
- Multi-langue (arabe / français) — interface en anglais dans le MVP ; internationalisation i18n React prévue en v2.
- API publique partenaires — les API REST Spring Boot sont internes, une API publique est hors périmètre sécuritaire du MVP.

11. KPIs — Indicateurs de Succès Mesurables

Les 4 indicateurs suivants seront vérifiés lors de la démonstration finale S12 sur la version déployée en production (DigitalOcean).

#	KPI	Valeur cible	Méthode de vérification
KPI-1	Temps de génération d'un contrat PDF	< 30 secondes	Démonstration en direct S12 — chronomètre
KPI-2	Temps de réponse API Spring Boot opérations courantes	< 2 secondes sur DigitalOcean en production	Mesure Postman / DevTools sur app déployée S12
KPI-3	Taux de livraison fonctionnalités Must Have (F1, F2, F3)	100 % des Must Have démontrables en S12	Revue des critères d'acceptation Jira S12

KPI-4	Rejet des tentatives de double réservation simultanée	100 % des conflits rejetés (test automatisé)	Test pipeline GitHub Actions — scénario UC-03 concurrent
--------------	---	--	--

12. Planification Macro — S1 à S12

Séances	Phase	Activités principales	Livrables
S1–S2	Initialisation & Planification	Fiche projet, Gantt, Mini-CDC, WBS, estimation COCOMO/Analogie	Fiche Projet, Gantt, Mini-CDC (S2)
S3–S5	Spécification	PERT + chemin critique, Product Backlog v1 Jira (19 US MoSCoW), architecture Sprint 0	PERT, Product Backlog Jira (S5)
S6–S8	Développement core (Sprint 0 + Sprint 1)	Architecture Spring Boot + PostgreSQL, modules F3 (Auth), F1 (Flotte), F2 (Réservation backend + portail client React)	Sprint Backlog, modules Auth + Flotte + Réservation
S9–S10	Intégration & CI/CD (Sprint 2)	F4 (Espace client), F4+ (Créances + relances), état des lieux numérique, pipeline CI/CD GitHub Actions + DigitalOcean	Dépôt Git structuré (S9), Pipeline CI/CD (S10)
S11–S12	Tests & Livraison finale (Sprint 3)	Tests end-to-end, test utilisateur gérant réel, corrections bugs, documentation technique, démonstration MVP	MVP v1 déployé sur DigitalOcean (S12)

13. Critères de Succès

Les 4 conditions ci-dessous sont vérifiables et mesurables lors de la démonstration finale S12.

#	Critère de succès	Vérification S12
CS-1	Toutes les fonctionnalités Must Have (F1 Flotte, F2 Réservation, F3 Auth) sont opérationnelles et démontrables en production sur DigitalOcean	Démonstration live complète
CS-2	Zéro double réservation possible — deux tentatives simultanées sur le même véhicule retournent une seule confirmation et un rejet HTTP 409	Test automatisé pipeline CI
CS-3	Le MVP est déployé et accessible en HTTPS sur DigitalOcean avec un pipeline CI/CD opérationnel (build + tests + déploiement automatique à chaque push sur main)	URL production accessible S12
CS-4	L'isolation multi-tenant est garantie — une agence connectée ne peut accéder aux données d'aucune autre agence (HTTP 403 retourné, validé par test automatisé)	Test d'isolation CI — HTTP 403 validé

14. Wireframes — Fonctionnalités Must Have

Les 3 wireframes ci-dessous couvrent les fonctionnalités Must Have prioritaires de REDr.
Structure uniquement — pas de design graphique final.

Wireframe 1 — Tableau de bord Flotte Agence (F1 Must Have)

Wireframe 2 — Portail Client : Recherche et Réservation en ligne (F2 Must Have)

Wireframe 3 — Interface Admin : Validation des comptes agences (F3 Must Have)

— Fin du Mini-Cahier des Charges REDr v1.0 — Mars 2026 —

Livrable 2 - Work Breakdown Structure (WBS) :

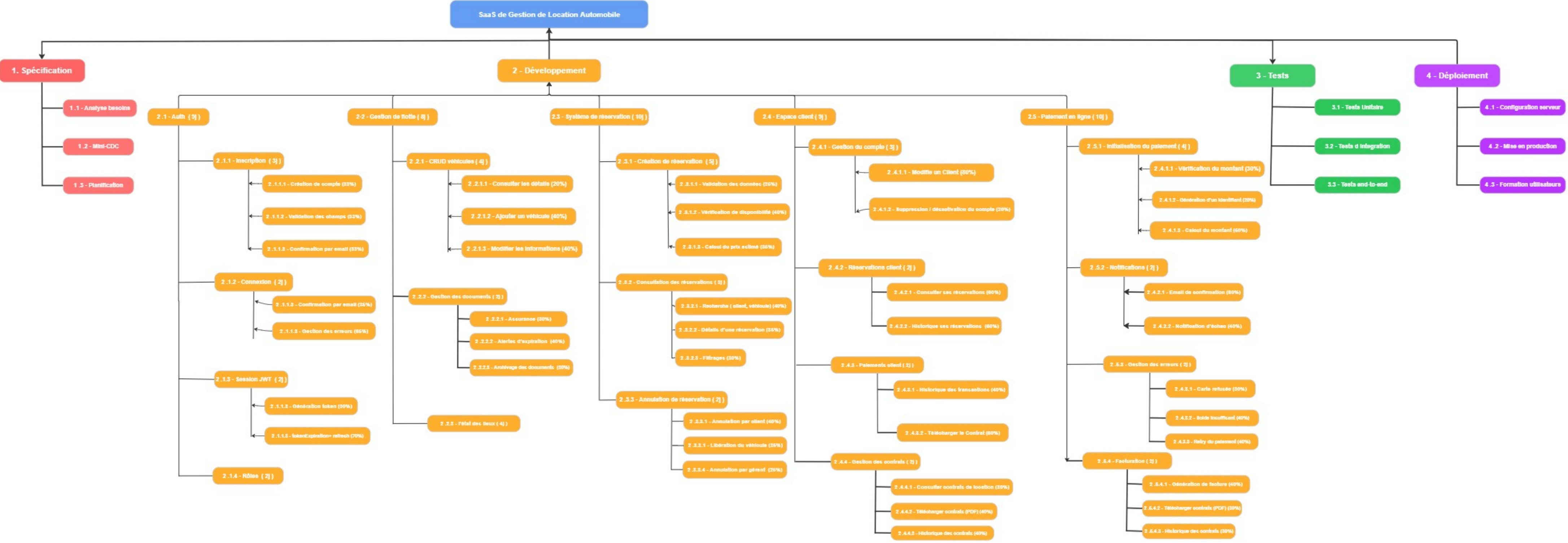
Niveau 0

Niveau 1

Niveau 2

Niveau 3

Niveau 4



DICTIONNAIRE WBS

SaaS de Gestion de Location Automobile

Code	Tache	Livrable	Responsable	Dure	Predecesseur	Critere de fin
2.1.1	Inscription	Module d'inscription fonctionnel avec formulaire de creation de compte valide et teste	Ali El Kheitte	3 jours	2.1 (Auth) - Le module Auth doit etre initialise avant de developper l'inscription	Les 3 sous-taches (creation de compte, validation des champs, confirmation par email) sont terminees et validees par le chef de projet
2.1.2	Connexion	Module de connexion securise avec gestion des erreurs et confirmation par email	Mohamed Talbi	2 jours	2.1.1 (Inscription) - L'inscription doit etre terminee pour disposer des comptes a authentifier	Connexion reussie avec token JWT retourne, gestion des erreurs fonctionnelle, tests unitaires >= 80%
2.1.3	Session JWT	Systeme de gestion des sessions JWT avec generation de token et invalidation a la deconnexion	Mohamed Talbi	2 jours	2.1.2 (Connexion) - La connexion doit etre operationnelle pour generer et valider les tokens JWT	Token JWT genere a la connexion, invalidation correcte a la deconnexion, renouvellement automatique fonctionnel
2.1.4	Roles	Systeme de gestion des roles utilisateurs (admin, client, gestionnaire) avec controle d'accès	Ashraf Bejjih	2 jours	2.1.3 (Session JWT) - Le systeme JWT doit etre en place pour associer les roles aux tokens	Roles crees et attribues correctement, acces aux ressources controle selon le role, tests de permissions valides
2.2.1	CRUD Vehicules	API complete de gestion des vehicules (creation, lecture, mise a jour, suppression)	Ashraf Bejjih	4 jours	2.1.3 (Session JWT) - L'accès aux operations CRUD doit etre securise via JWT	Les 3 sous-taches (consulter, ajouter, modifier) validees, reponses HTTP correctes, couverture tests >= 80%
2.2.2	Gestion des documents	Module de gestion des documents de vehicules (annonces, contrats, historique)	Mohamed Talbi	2 jours	2.2.1 (CRUD Vehicules) - Les vehicules doivent exister en base avant de gerer leurs mouvements	Annonces creees, expirations gerees automatiquement, historique des mouvements trace et consultable
2.2.3	Etat des lieux	Module d'etat des lieux des vehicules avec formulaire de saisie et stockage des donnees	Ali El Khiette, Ashraf Bejjih	4 jours	2.2.1 (CRUD Vehicules) - Les vehicules doivent etre enregistres pour effectuer un etat des lieux	Etat des lieux enregistre en BDD, photos et commentaires stockes, formulaire valide sans erreur
2.3.1	Creation de reservation	Formulaire de creation de reservation avec validation des donnees, verification de disponibilite et calcul du prix estime	Ali El Khiette, Ashraf Bejjih	5 jours	2.2.1 (CRUD Vehicules) - Des vehicules doivent etre disponibles pour etre reserves	Reservation enregistree en BDD, disponibilite confirmee, prix estime affiche, formulaires valides sans erreur
2.3.2	Consultation des reservations	Interface de consultation des reservations avec filtres par statut, date, vehicule et client	Mohamed Talbi	3 jours	2.3.1 (Creation de reservation) - Des reservations doivent exister pour pouvoir les consulter	Liste des reservations affichee correctement, filtres fonctionnels, details d'une reservation accessibles
2.3.3	Annulation de reservation	Module d'annulation de reservation avec gestion des regles metier et remboursement	Ashraf Bejjih	2 jours	2.3.1 (Creation de reservation) - Une reservation doit exister pour pouvoir etre annulee	Annulation enregistree, statut mis a jour, remboursement declenche si applicable, vehicule de remplacement propose

Code	Tache	Livrable	Responsable	Dure	Predecesseur	Critere de fin
2.4.1	Gestion du compte	Interface de gestion du compte client (modification du profil, suppression/desactivation)	Ali El Khiette	3 jours	2.1.1 (Inscription) - Un compte doit etre cree avant de pouvoir le gerer	Mise a jour du profil sauvegardee avec succes, desactivation fonctionnelle, acces securise par token JWT
2.4.2	Reservations client	Espace client avec historique et suivi des reservations passees et en cours	Ali El kheitte	2 jours	2.4.1 (Gestion du compte) et 2.3.1 (Creation reservation) - Le compte et les reservations doivent exister	Historique des reservations affiche, filtres par date/statut fonctionnels, details consultables
2.4.3	Paiements client	Espace client de gestion des paiements avec historique des transactions et telechargement	Ali El kheitte	2 jours	2.5.1 (Initialisation de paiement) - Le module de paiement doit etre operationnel pour afficher l'historique	Historique des paiements affiche correctement, contrats telechargeables, statuts des transactions a jour
2.4.4	Gestion des contrats	Module de gestion des contrats de location avec generation PDF et signature electronique	Ashraf Bejjih	2 jours	2.3.1 (Creation de reservation) - La reservation doit etre confirmee avant de generer un contrat	Contrat genere en PDF, signe electroniquement, stocke en BDD, consultable par client et gestionnaire
2.5.1	Initialisation de paiement	Integration de la passerelle de paiement (Stripe/PayPal) avec verification du montant et declenchement de la transaction	Mohamed Talbi	4 jours	2.3.1 (Creation de reservation) - Le montant a payer est calcule lors de la reservation	Transaction initialisee, montant verifie, token de paiement genere, integration testee en sandbox
2.5.2	Notifications	Systeme de notifications email/SMS pour les evenements cls (confirmation, rappels, erreurs)	Ashraf Bejjih	2 jours	2.5.1 (Initialisation de paiement) - Les notifications sont declenchees apres un paiement reussi ou echoue	Notifications envoyees dans les 2 min, modeles email/SMS valides, logs d'envoi consultables
2.5.3	Gestion des erreurs	Module de gestion des erreurs de paiement avec codes de retour, retry et notification d'echec	Ashraf Bejjih	2 jours	2.5.1 (Initialisation de paiement) - Les erreurs ne peuvent etre gerees qu'apres l'initialisation du flux de paiement	Codes d'erreur definis, retry automatique en cas d'echec reseau, notification client en cas d'echec
2.5.4	Facturation	Module de generation et d'envoi des factures PDF avec numerotation automatique et export comptable	Mohamed Talbi	2 jours	2.5.1 (Initialisation de paiement) - La facture est generee apres confirmation du paiement	Facture generee automatiquement, numerotation sequentielle, envoi par email, export CSV pour comptabilite
3.1	Tests Unitaires	Rapport de tests unitaires couvrant tous les modules developpes (Auth, Vehicules, Reservation, Paiement)	Tous l'équipe	5 jours	2.1, 2.2, 2.3, 2.4, 2.5 (tous modules) - Les modules doivent etre developpes avant d'etre testes unitairement	Couverture de code >= 80% par module, 0 test en echec, rapport genere et valide par le chef de projet
3.2	Tests d'integration	Rapport de tests d'integration couvrant les interactions entre les modules Auth, Reservation, Paiement et Espace client	Tous l'équipe	5 jours	3.1 (Tests Unitaires) - Les tests unitaires doivent etre valides avant de tester les interactions entre modules	Taux de reussite >= 95%, tous les flux critiques (reservation-paiement, auth-profil) valides sans regression
4.1	Configuration serveur	Infrastructure cloud configuree et securisee (serveur, BDD, reseau, DNS, SSL)	Tous l'équipe	3 jours	3.3 (Tests end-to-end) - L'application doit etre validee avant de configurer l'environnement de	Serveur operationnel, HTTPS active, base de donnees securisee, DNS configure

Code	Tache	Livrable	Responsable	Dure	Predecesseur	Critere de fin
					production	
4.2	Mise en production	Application SaaS deployee en production, accessible, securisee et monitoree	Tous l'équipe	4 jours	4.1 (Configuration serveur) - Le serveur doit etre configure avant de deployer l'application en production	Application disponible a 100%, HTTPS active, pipeline CI/CD operationnel, monitoring configure, rollback possible
4.3	Formation utilisateurs	Sessions de formation et guide utilisateur destines aux gestionnaires et administrateurs	Ali El Khiette	2 jours	4.2 (Mise en production) - L'application doit etre en production pour former les utilisateurs sur l'environnement reel	Sessions de formation realisees, guide utilisateur livre, questionnaire de satisfaction rempli

Note : Les durees estimees sont indicatives et peuvent etre ajustees selon les contraintes du projet. Les predecesseurs indiques correspondent aux codes WBS de reference.

Livrable 3 Estimation du projet:

1) - Analogie :

Principe :

1 - Identifie les différence

2 - Donner un coefficient (si plus simple <1 | si identique = 1 | plus si complexe > 1) Saas RentCar multi-Agence :

Fonctionalte (Automobile) :

F1 — Authentification multi-roles (Client / Agence / Admin)

F2 — Gestion des flotte

F3 — Système de réservation

F4 — Espace client

F5 — Paiement en ligne

Plateforme de réservation d'hotel :

Hypothes réalise , projet déjà réalisé = 5 mois .

RentCar ---> Hotel Booking

Véhicule ---> Chambre

Réservation ---> Réservation

Disponibilité ---> Disponibilité

Paiement ---> Paiement

Contrat PDF ---> Facture PDF

Fonctionalte :

F1 — Authentification multi-roles (client , Admin)

F2 — Gestion des chambres

F3 — Système de réservation

F4 — Espace client

F5 — Paiement en ligne

Évaluation des différences fonctionnelles et de leur impact (COCOMO) :

Différence identifiée	Impact	Coefficient	Pourquoi?
Authentification	plus grand	1,1	il ya aussi gestion des Agences (10%)
Gestion des flotte	plus grand	1,15	plus d'états , conflits , de calculs (20%)
Système de réservation	plus grand	1,2	La gestion des options (GPS, assurance...) (20%)
Espace client	plus grand	1,15	voir les détails (prix, durée, véhicule) (15%)
Paiement en ligne	plus grand	1,3	caution (deposit) ,pénalités éventuelles (30%)

Résutat:

$D_{\text{projet}} = 3 \text{ mois} * 1,1 * 1,15 * 1,2 * 1,15 * 1,3$

$D_{\text{projet}} = 11,34 \text{ mois}$

2) - COCOMO basique (pro l organique) :

2-1) -

Module	estimation	justification
Authentification (F1)	2500	sécurité + Login (username/password) + Inscription
Gestion flotte (F2)	4000	CRUD véhicules + statuts + validation des champs
Réservation (F3)	4500	generation d une contat + logique complexe
Espace client (F4)	2500	validation des champs + validation des champs + CRUD
Paieement (F5)	2000	Gestion des (statuts de paiement + statuts de paiement)

Résutat :

Total = 17500 lignes

KLOC = 17,5 ---> 18

2-2) - Application des formules COCOMO :

Mode choisi : **organique**

$PM = 2.4 * (15) ^ 1.05 = 49,9 \text{ --> } 50 \text{ mois-homme}$

$TDEV = 2.5 * PM ^ 0.38 = 11.05 \text{ ---> } 11,1 \text{ mois}$

$P = PM/TDEV = 50/11,1 = 4,5$

Donc 5 développeurs recommandés

2-3) - Comparaison avec l'équipe réelle

mon equipe est constitue avec 3 dev

mais en a besoin 5 dev

donc mon equipe est petite pour ces tache

2-4) - Analyse de l'écart :

Délai estimé (COCOMO) : $\approx 11,1 \text{ mois}$

Délai imposé : $\approx 11,34 \text{ mois}$ (ton calcul D_{projet})

Équipe recommandée : 5 dev

Équipe réelle : 3 dev

Option 1 : Réduction du périmètre (RECOMMANDÉ)

Supprimer ou simplifier :

Païement avancé → garder paiement simple

Logique complexe réservation → version basique

Moins de KLOC

Moins de PM

Compatible avec 3 dev

Option 2 : Augmenter l'équipe

Passer de 3 → 5 dev

pour resoudre cet ecart, plusieurs strategie peuvent éter adopte :

la réduction du périmètre fonctionnel,

la parallélisation des tâches,

ou l'utilisation de frameworks pour diminuer l'effort de développement.

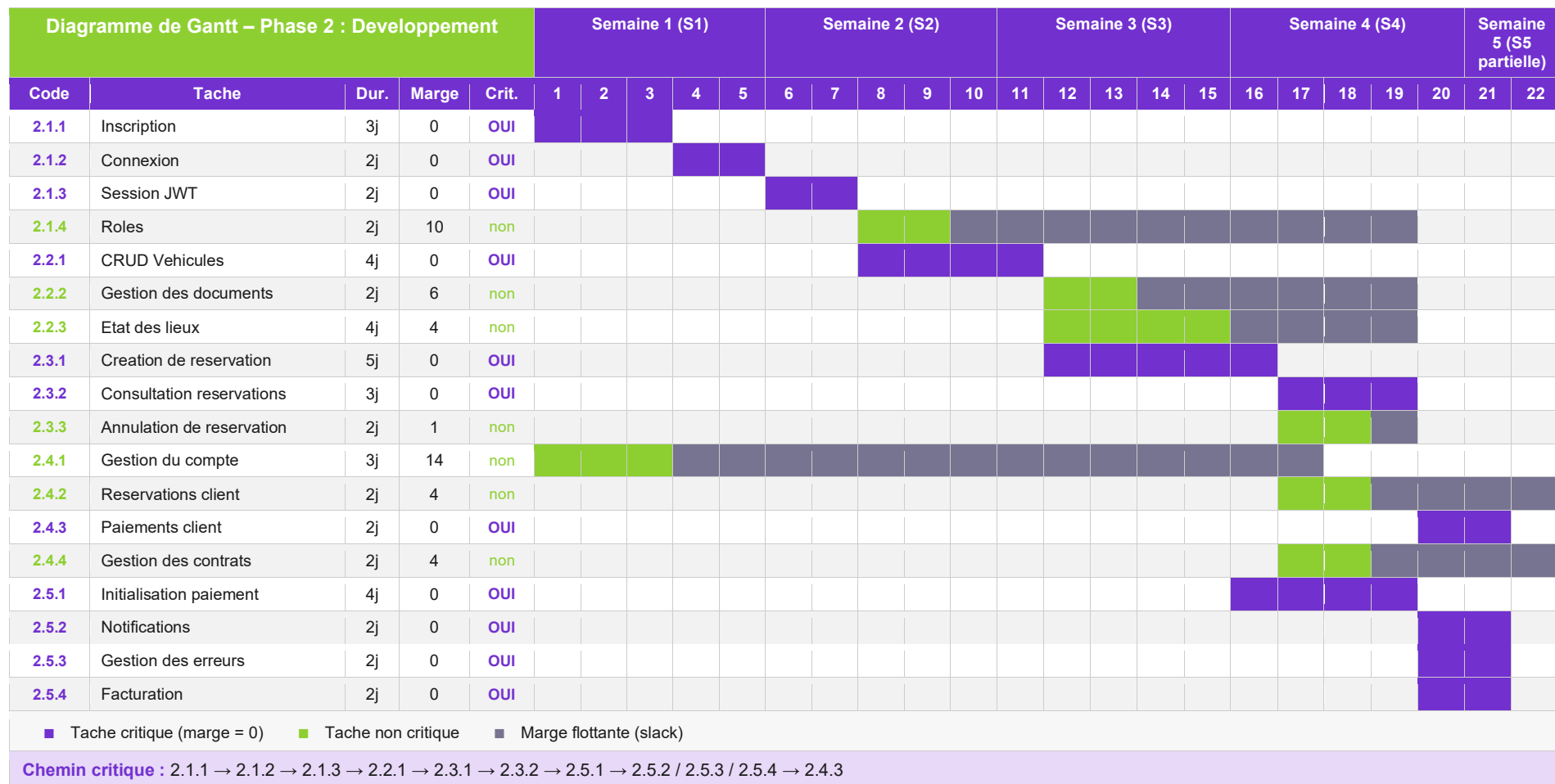
Livrable 4 - Diagramme de Gantt :

code wbs	Tâche	Durée	Début tôt	Fin tôt	Debut tard	Marge total	Critique ?
2.1.1	Inscription	3 jours	0 (S1)	3 (S1)	0 (S1)	0	.oui
2.1.2	Connexion	2 jours	3 (S1)	5 (S2)	3 (S1)	0	oui
2.1.3	Session JWT	2 jours	5 (S2)	7 (S2)	5 (S2)	0	oui
2.1.4	Rôles	2 jours	7 (S2)	9 (S2)	17 (S4)	10	non
2.2.1	CRUD Véhicules	4 jours	7 (S2)	11 (S3)	7(S2)	0	oui
2.2.2	Gestion des documents	2 jours	11 (S3)	13 (S3)	17 (S4)	6	non
2.2.3	État des lieux	4 jours	11 (S3)	15 (S3)	15 (S4)	4	non
2.3.1	Création de réservation	5 jours	11 (S3)	16 (S4)	11 (S3)	0	oui
2.3.2	Consultation des réservations	3 jours	16 (S4)	19 (S4)	16 (S4)	0	oui
2.3.3	Annulation de réservation	2 jours	16 (S4)	18 (S4)	17 (S4)	1	non
2.4.1	Gestion du compte	3 jours	3 (S1)	6 (S2)	17 (S4)	14	non
2.4.2	Réservations client	2 jours	16 (S4)	18 (S4)	20 (S5)	4	non
2.4.3	Paielements client	2 jours	20 (S5)	22 (S5)	20 (S5)	0	oui
2.4.4	Gestion des contrats	2 jours	16 (S4)	18 (S4)	20 (S5)	4	non
2.5.1	Initialisation de paiement	4 jours	16 (S4)	20 (S5)	16 (S4)	0	oui
2.5.2	Notifications	2 jours	20 (S5)	22 (S5)	20 (S5)	0	oui
2.5.3	Gestion des erreurs	2 jours	20 (S5)	22 (S5)	20 (S5)	0	oui
2.5.4	Facturation	2 jours	20 (S5)	22 (S5)	20 (S5)	0	oui

DIAGRAMME DE GANTT

SaaS de Gestion de Location Automobile

Phase 2 : Developpement – Livrable 4



Note : Les durees sont en jours ouvrables. La marge indique le nombre de jours de flottement avant que la tache ne devienne critique.